

Предварительный план выполнения задания

OCR.1 Разведка движков: Tesseract vs EasyOCR vs PaddleOCR

Цель: Сравнить три движка на репрезентативном датасете и выбрать лучший для разных сценариев.

1. **Подготовка инфраструктуры:**
 - Установить Tesseract 5 через apt и Python-обертку (pytesseract).
 - Установить EasyOCR и PaddleOCR через pip.
 - Убедиться, что все зависимости (OpenCV, PyTorch для EasyOCR) установлены.
2. **Сбор датасета (50 изображений):**
 - Создать 5-6 категорий по 8-10 изображений: машинописный текст (рус/англ), рукописный текст, отсканированные таблицы, документы с низким качеством, изображения с текстом поверх рисунков, смешанные языки.
 - Для каждой картинки создать эталонный файл .txt с правильным текстом (для расчета CER/WER).
3. **Разработка бенчмарк-скрипта:**
 - Написать Python-скрипт, который для каждого движка выполняет:
 - Прогон всех 50 изображений.
 - Замер времени выполнения (time.perf_counter()).
 - Расчет CER (символьная ошибка) и WER (словесная ошибка) с использованием библиотеки jiwer.
4. **Сбор и оформление результатов:**
 - Свести результаты в таблицу: **Движок | Средний CER | Средний WER | Среднее время (сек).**
 - Разбить таблицу по категориям (например, точность на таблицах).
 - **Вывод:** Написать рекомендацию (например: *"Tesseract — для печатного текста, EasyOCR — хорош для рукописи, PaddleOCR — лучший для таблиц и смешанных языков"*).

OCR.2 Пайплайн предобработки и батчевая обработка

Цель: Создать инструмент для массовой обработки изображений с использованием всех ядер CPU.

1. Реализация пайплайна предобработки (OpenCV):

- Написать функцию `preprocess_image()`:
 - `grayscale` (преобразование в оттенки серого).
 - `otsu_threshold` (бинаризация).
 - `deskew` (коррекция перекоса через определение угла поворота).
 - `denoise` (применение Non-local Means Denoising).
 - `dpi_normalization` (приведение всех изображений к единому разрешению, например, 300 DPI).

2. Реализация CLI-инструмента (`ocr_batch.py`):

- Использовать `argparse` для парсинга аргументов: `--input`, `--output`, `--workers`, `--engine` (для выбора движка из OCR.1).
- Реализовать `multiprocessing.Pool`: функция-воркер принимает путь к файлу, применяет `preprocess_image()`, вызывает выбранный OCR-движок и сохраняет результат в структуру.

3. Тестирование производительности (Speedup):

- Замерить время обработки 100 изображений с 1 воркером.
- Замерить время с N воркерами (где N = количество ядер CPU).
- Построить график или таблицу " $\text{Speedup} = T(1) / T(N)$ ".

4. Выход: Оформить скрипт, таблицу и примеры JSON-вывода.

OCR.3 FastAPI-сервис с очередью задач (Celery + Redis)

Цель: Построить масштабируемый асинхронный микросервис для обработки тяжелых документов.

1. Настройка окружения:

- Написать `docker-compose.yml` для поднятия 4-х сервисов:
 - `api` (FastAPI + Uvicorn).
 - `worker` (Celery с очередью `ocr_tasks`).
 - `redis` (брокер сообщений и бекенд для хранения результатов).
 - `flower` (мониторинг очереди).

2. Разработка FastAPI:

- `POST /ocr/submit`: Принимает файл (`UploadFile`), генерирует `task_id`, ставит задачу в Celery (`process_document.delay(file_path)`). Возвращает `{"task_id": "..."}`. Сохраняет файл во временную папку.
- `GET /ocr/status/{task_id}`: Возвращает статус (PENDING, PROCESSING, SUCCESS, FAILURE), запрашивая состояние у Celery (`AsyncResult`).
- `GET /ocr/result/{task_id}`: Возвращает JSON с результатами распознавания.

3. Celery-воркер:

- Создать файл `tasks.py`.
- Определить задачу, которая принимает путь к файлу, вызывает функцию распознавания из OCR.2, и возвращает структурированный ответ.

4. Тестирование:

- Написать интеграционный тест (pytest), который:
 - Отправляет запрос на submit.
 - Ждет завершения через status.
 - Проверяет корректность result.

5. Документация: Убедиться, что Swagger (/docs) доступен и показывает все эндпоинты.

OCR.4 Fine-tuning Tesseract на специфичных документах

Цель: Улучшить качество распознавания на узкоспециализированных данных.

1. Сбор данных и разметка (Label Studio):

- Развернуть Label Studio (через Docker). Подключить volume для сохранения проектов.
- Загрузить минимум 50 изображений одного типа.
- Создать проект по разметке текста (OCR Annotation). Разметить текст вручную для каждого изображения (Boxes -> Text).
- Экспортировать данные в формате, понятном для Tesseract (например, .gt.txt для каждого изображения).

2. Подготовка данных для Tesstrain:

- Конвертировать изображения и размеченные тексты в формат .tif и .box.
- Использовать tesseract для создания файлов .tr (тренировочные) для каждого изображения.

3. Процесс Fine-tuning:

- Установить tesstrain.
- Скачать базовую модель (например, eng или rus).
- Запустить обучение на 50+ изображениях (команда: tesstrain.sh --model_name custom ...).
- Получить файл custom.traineddata.

4. Валидация:

- Загрузить тестовую выборку (из тех же 50, но не участвовавшую в обучении, или дополнительную).
- Сравнить CER/WER базовой модели и новой (custom). Подсчитать прирост точности (%).

OCR.5 Классический OCR vs Qwen VL

Цель: Сравнить традиционные движки с современной VLM (Vision Language Model) и определить зоны ответственности.

1. Развертывание Qwen2.5-VL-7B:

- Установить transformers, accelerate, bitsandbytes.
- Написать скрипт-загрузчик модели с квантизацией (для экономии VRAM) или полной точности (FP16).
- Написать функцию промпта: "Extract all the text from this image".

2. Запуск сравнения (qwen_vs_ocr.py):

- Взять тот же датасет из 50 изображений.
- Прогнать через Qwen VL (замерить время инференса).
- Прогнать через "лучший" классический движок из OCR.1 (с предобработкой).

OCR.6 Нагрузочное тестирование и мониторинг сервиса

Цель: Проверить сервис под давлением и найти узкие места.

1. Настройка мониторинга:

- Добавить в docker-compose.yml сервисы prometheus и grafana.
- Подключить prometheus-fastapi-instrumentator к FastAPI для сбора метрик (количество запросов, время ответа).
- Настроить сбор метрик из Celery (Flower API или экспортер).
- Настроить Node Exporter для сбора метрик CPU/RAM самой VM.

2. Настройка Grafana:

- Импортировать готовые дашборды для Node Exporter и FastAPI.
- Создать простой дашборд для очереди Celery (количество задач в pending).

3. Написание Locust-теста:

- Создать файл locustfile.py.
- Описать поведение пользователя: загрузить PDF на 5 страниц -> подождать статус -> проверить результат.
- Запустить тесты с профилями нагрузки: 10, 50, 100 одновременных пользователей.

4. Анализ результатов:

- Зафиксировать Throughput (задач/сек).
- Зафиксировать Latency (P50, P95).
- Определить момент, когда очередь начинает расти в бесконечность (точка насыщения).
- **Узкое место:** Описать, что именно стало лимитирующим (CPU воркера, скорость диска, сеть, GPU для Qwen).

5. Рекомендации: Предложить варианты масштабирования (горизонтальное: добавляем еще один воркер; вертикальное: увеличиваем RAM/CPU).

OCR.7 Итоговый отчёт и документация production-сервиса

Цель: Собрать все компоненты в единый пакет для успешной сдачи проекта.

1. **README.md:**
 - Написать инструкцию "Deploy in one command" (например, `git clone + docker-compose up -d`).
 - Описать переменные окружения.
2. **Архитектурная схема:**
 - Нарисовать блок-схему: Пользователь -> FastAPI -> Redis -> Celery Worker -> OCR движок.
 - Указать стрелками потоки данных и точки масштабирования (где можно увеличить количество воркеров).
3. **Сводная аналитика:**
 - Вставить таблицы из пунктов OCR.1, OCR.4, OCR.5.
 - Вставить графики из нагрузочных тестов (OCR.6).
4. **Runbook (Руководство по эксплуатации):**
 - Описать 5 инцидентов:
 1. *Очередь зависла* -> Команда: `celery purge -f` и рестарт.
 2. *Воркер упал* -> Команда: перезапуск контейнера + проверка логов.
 3. *Redis недоступен* -> Проверить `redis-cli ping`, перезапустить `compose`.
 4. *CPU OOM (Out of Memory) при Qwen VL* -> Уменьшить размер батча или использовать квантизацию (4-bit).
 5. *Закончилось место на диске* -> Команда очистки старых файлов.
 - Для каждого инцидента дать четкие команды диагностики (`docker logs worker | grep ERROR` и т.д.).
5. **Дальнейшее развитие (Future Roadmap):**
 - Описать гибридный пайплайн: маршрутизатор, который анализирует сложность документа (по размеру, по плотности текста) и отправляет простые документы в Tesseract, а сложные в Qwen VL, экономя ресурсы.